НИЯУ МИФИ

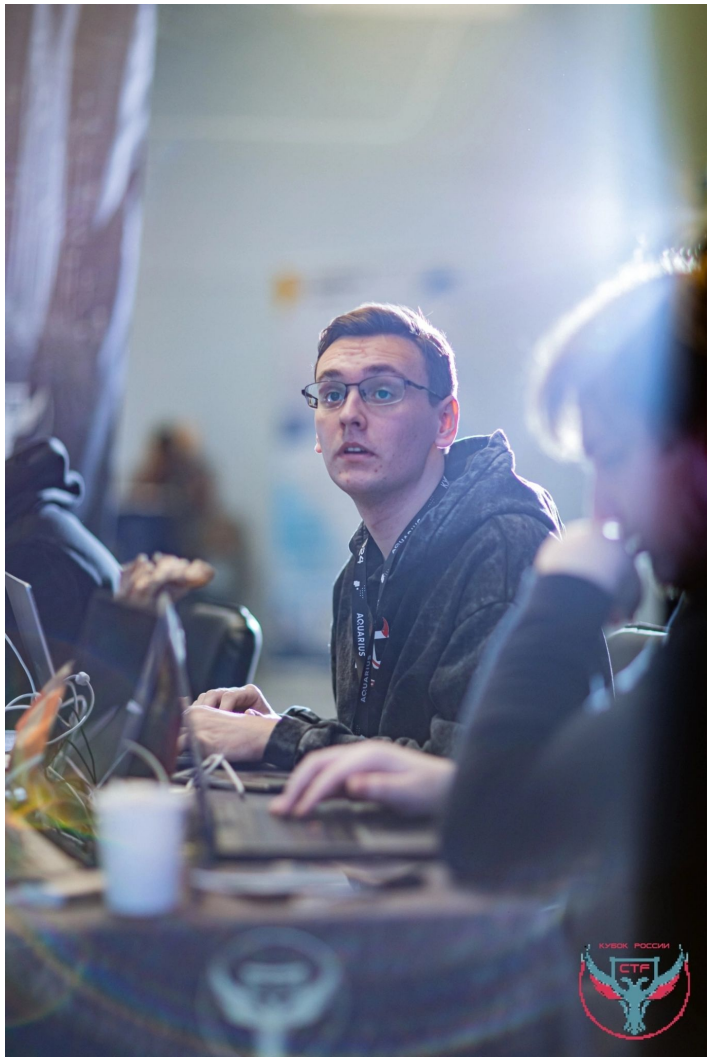
Бинарные уязвимости

Модуль 1. Введение в реверс-инжиниринг

Дата

$whoami

- Рудаков Даниил (@danissimurr)
- Реверс-инженер в ООО “Код безопасности”
- LCD, Mephi CTF
- Автор задач для школьных и студенческих CTF
- Преподаватель на сменах по ИБ



План занятия

НИЯУ МИФИ

Введение в реверс-инжиниринг

1. Введение. Основные подходы. Применение
2. Реверс исходного кода
3. Реверс Python, C#, Java
4. Реверс исполняемых файлов
5. Итоги

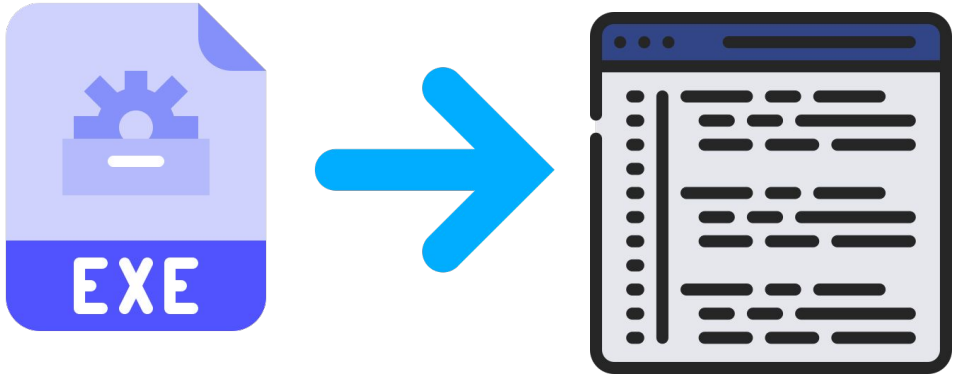
Введение. Основные подходы. Применение

Что такое реверс-инжиниринг?

НИЯУ МИФИ

Реверс-инжиниринг

Исследование устройства или программы (часто без исходного кода), чтобы восстановить логику работы и, при необходимости, изменить поведение



С чего начать изучение реверс-инжиниринга

Основные подходы

Статический анализ

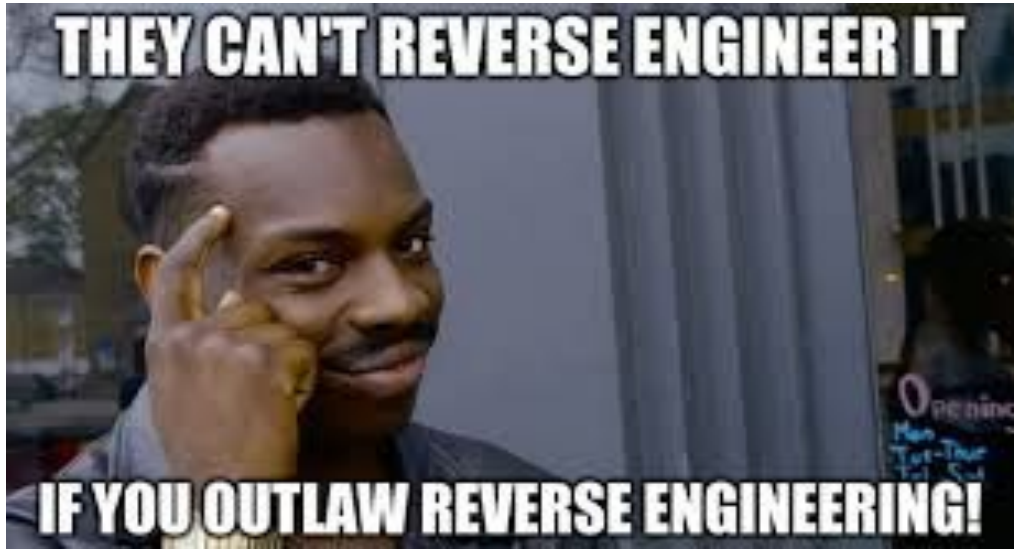
1. Получаем ассемблерный или исходный код
2. Исследуем код без запуска

Динамический анализ

1. Запускаем программу в отладчике
2. Исследуем, что происходит на каждом шаге (инструкции, память)

Применение реверс-инжиниринга

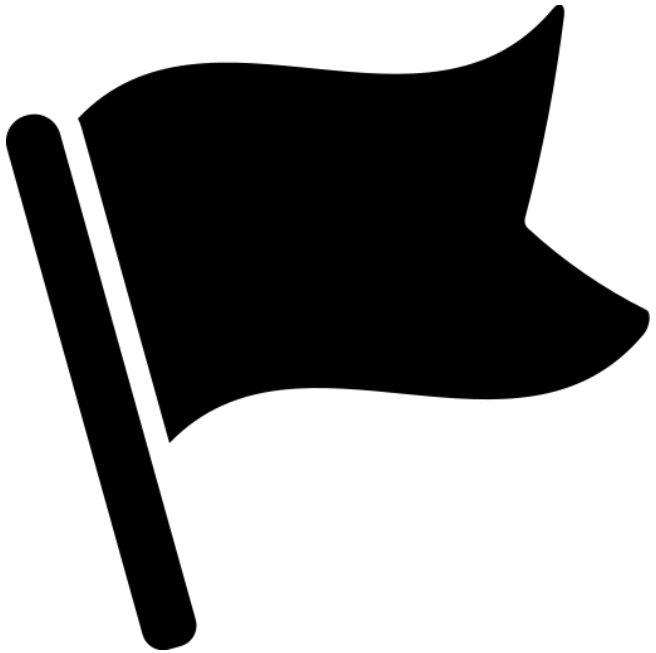
1. Исследование вредоносного ПО
   a. Вредоносное ПО: исследование поведения и каналов распространения
   b. Классификация вредоносных программ | Блог Касперского
2. Пентест
3. Посмотреть, как работает продукт конкурента
4. CTFы

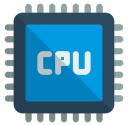


Право на реверс. Как обратная разработка выглядит с юридической точки зрения — Хакер

Типичная задача категории Reverse

1. Дана программа (исходный код, байт-код, исполняемый файл...), которая запрашивает пароль - **crackme**
2. Нужно исследовать, как проверяется пароль
3. На основе этого нужно получить пароль, который программа примет
4. Правильный пароль - ответ (флаг, обычно в формате **flag{...}**)





МАШИННЫЙ КОД

Опкоды
(номера команд)

Машинный код

1. Интерпретируется непосредственно процессором
2. Содержится в исполняемых файлах
3. Можно посмотреть с помощью hex-редактора
4. Программистам неудобно писать программы в виде чисел

Машинный код — Википедия

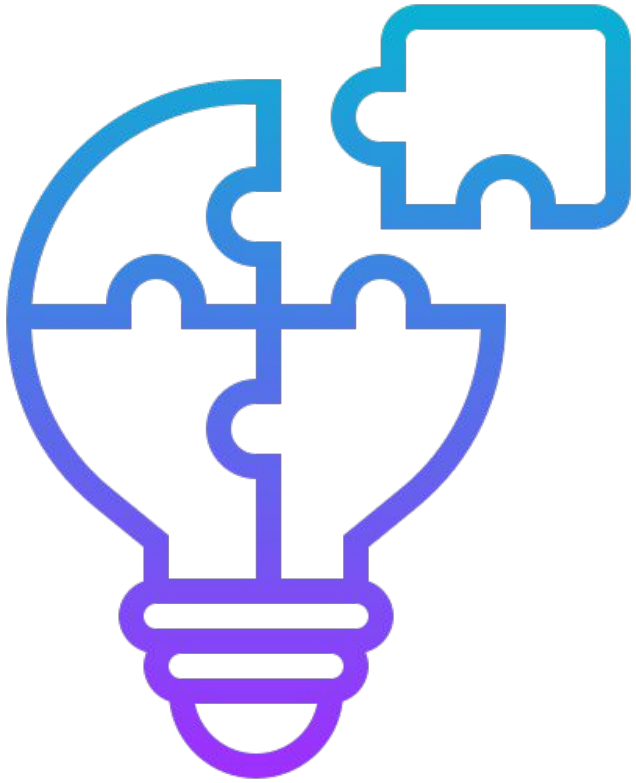
Решение

Каждой инструкции даём мнемонику

1. Сложение (№1) - **add**
2. Вычитание (№2) - **sub**
3. Сравнение (№3) - **cmp**
4. Перемещение в памяти (№4) - **mov**

Теперь сложение (2+2) вместо
1 2 2
выглядит как
add 2, 2

Так появился язык ассемблера



List of x86 instructions - Wikipedia

intel
AMD
ЯЗЫК АССЕМБЛЕРА
Мнемоники команд
Ассемблер
Дизассемблер
CPU
EXE
МАШИННЫЙ КОД
Опкоды (номера команд)

Язык ассемблера

- Мнемоники команд проще читать и писать
- **Ассемблер** конвертирует ассемблерный код в машинный код
- **Дизассемблер** конвертирует машинный код в ассемблерный
- Основные ассемблеры на Linux: nasm, yasm, fasm, gas
- 2 основных синтаксиса: Intel, AT&T
- Наиболее удобные дизассемблеры: IDA, Ghidra, Radare2

1.asm

```
1  section .data
2      text db "Hello world!", 10
3
4  section .text
5      global _start
6
7  _start:
8      mov rax, 1
9      mov rdi, 1
10     mov rsi, text
11     mov rdx, 14
12     syscall
13
14     mov rax, 60
15     mov rdi, 0
16     syscall
```

Ассемблер: рассматриваем каждый байт «Hello, World!». Как на самом деле работают программы на уровне процессора и ОС / Хабр

Проблема

- Архитектур много: X86, X86_64, ARM, MIPS...
- У каждой архитектуры свой набор операций
- Нужно писать отдельную программу под каждую архитектуру
- **Решение** - создать универсальный язык программирования высокого уровня

НИЯУ МИФИ

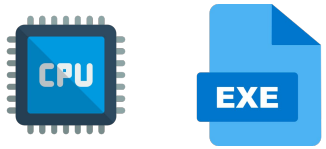
ЯП ВЫСОКОГО УРОВНЯ

Понятный высокоуровневый код (циклы, классы...)

ЯЗЫК АССЕМБЛЕРА

Мнемоники команд

Ассемблер **Дизассемблер**



МАШИННЫЙ КОД

Опкоды (номера команд)

Проблема 2

- Компьютер не понимает ЯП высокого уровня
- Нужно представить этот код на языке ассемблера
- Хочется это делать автоматически под каждую архитектуру

Возможные решения

Компиляция

Перезапись **компилятором** высокоуровневых команд с помощью эквивалентного набора команд языка ассемблера
(цикл -> jmp)

Интерпретация

Отдельная программа - **интерпретатор** - исполняет высокоуровневый код

НИЯУ МИФИ

ЯП ВЫСОКОГО УРОВНЯ → Понятный высокоуровневый код (циклы, классы...)

Компилятор/интерпретатор ↓ ↑ Декомпилятор

ЯЗЫК АССЕМБЛЕРА → Мнемоники команд

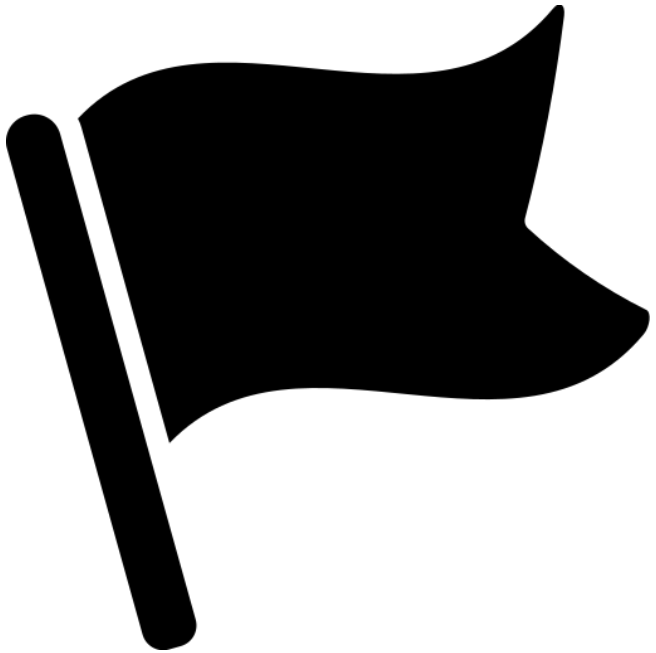
Ассемблер ↓ ↑ Дизассемблер

МАШИННЫЙ КОД → Опкоды (номера команд)

Что можно ревёрсить?

1. Исходный код
2. Байт-код (Python, Java, C#...)
3. Исполняемые файлы - бинари (C, C++, Rust)

И многое другое...



Реверс исходного кода

Реверс исходников

Что может быть

Код на любом ЯП, в том числе bash-скрипты и powershell-скрипты

Тренировать здесь

Как решать

- Смотрим документацию по ЯП
- Мучаем нейронку
- Запускаем программу или её часть
- Модифицируем программу (отладочный вывод...)

Реверс Python, C#, Java

Что такое байт-код?

- **Байт-код** - промежуточный код между исходным кодом и машинным кодом
- Содержит машиноподобные инструкции, которые понимает виртуальная машина. Процессор не выполняет эти инструкции

Байт-код — Википедия

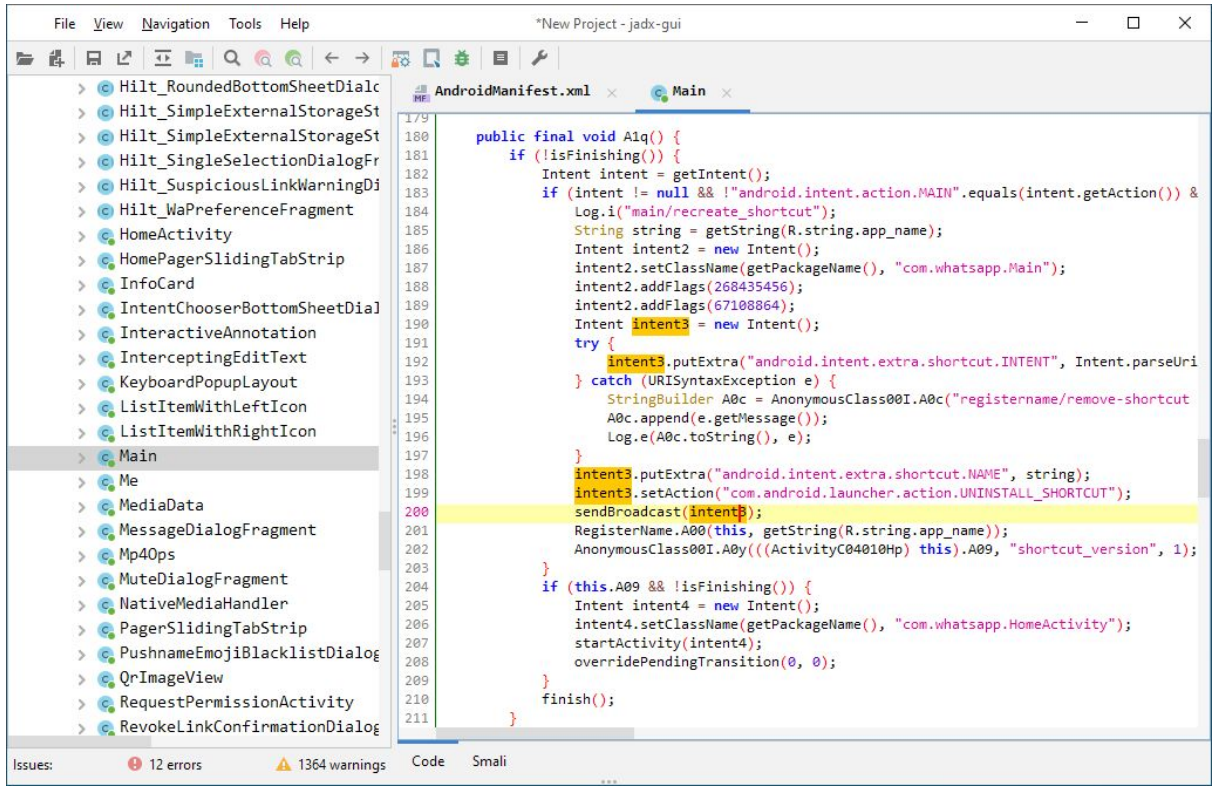
Байт-код VS машинный код

Байт-код	Машинный код
Работает в ВМ	Исполняется процессором
Возможен почти полный возврат к исходному коду	Полный возврат невозможен
Сохраняются структуры классов, функций, имена (если не было обфускаций)	Имена и структуры могут быть потеряны
Просто анализировать	Сложнее анализировать

Разница между машинным кодом и байт-кодом / Ravesli

Реверс Java

- **Инструмент** - jadx
- Наиболее часто встречается:
  - **.class** - один файл с байт-кодом
  - **.jar** - Java Archive
  - **.apk** - для Android



Реверс C#

- **Инструмент** - dnSpy
- Наиболее часто встречается:
  - **.exe** - сам бинарь
  - **.dll** - библиотека

Program

```
1   using System;
2   using System.Text;
3
4   namespace ConsoleApp21
5   {
6       // Token: 0x02000002 RID: 2
7       class Program
8       {
9           // Token: 0x06000001 RID: 1 RVA: 0x00002050 File Offset: 0x00000250
10          static void Main(string[] args)
11          {
12              StringBuilder sb = new StringBuilder(args.Length * 100);
13              foreach (string s in args)
14              {
15                  sb.Append(s);
16                  sb.AppendLine();
17              }
18              Environment.FailFast(sb.ToString());
19          }
20
21          // Token: 0x06000002 RID: 2 RVA: 0x00002097 File Offset: 0x00000297
22          public Program()
23          {
```

100 %

Locals

Name	Value	Type
args	{string[0x0000_0000]}	string[]
sb	null	System.Text.StringBuilder
i	0x0000_0000	int
s	null	string

Exception Settings | Module Breakpoints | Breakpoints | Bookmarks | Locals | Autos | Watch 1 | Call Stack | Processes | Modules | Threads | Memory 1 | Output

Реверс Python

- **decompile3** - для очень старых версий
- **uncompyle6** - для старых версий
- **pycdc** - хорош до примерно 3.9
- pylingual **(веб-версия)** - декомпилятор здорового человека
- pylingual **(локальная версия)** - долго ставить, но хорошо работает, когда веб-версия недоступна
- Наиболее часто встречается: **.pyc**

PYLINGUAL: Toward Perfect Decompilation of Evolving High-Level Languages

Реверс бинарей

Основные архитектуры процессоров

- **X86 и X86_64** - Intel, AMD - CISC (Complex Instruction Set Computing)
- **ARM** - Apple, Samsung, Qualcomm, Nvidia - RISC (Reduced Instruction Set Computing)
- **MIPS** - RISC
- ...

У каждой архитектуры свой набор команд
Далее будем рассматривать **X86_64**

CISC, RISC и другие типы микропроцессоров OTUS

Основные форматы исполняемых файлов

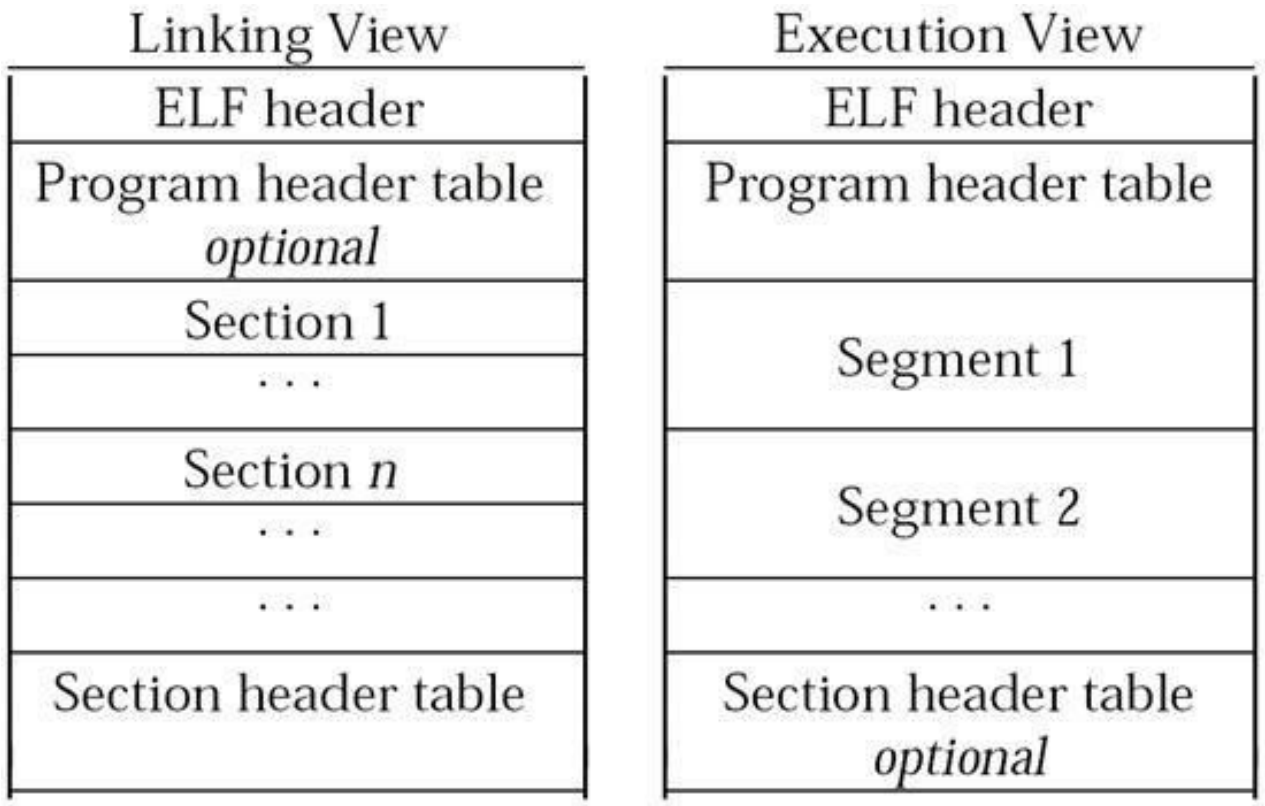
- Windows - **PE (Portable Executable):** .exe, .dll, .sys...
- Linux - **ELF (Executable Linking Format)**
- macOS - **Mach-O**
- ...

Структура ELF

ELF - по сути контейнер в котором лежат код, данные, информация о загрузке и тд.

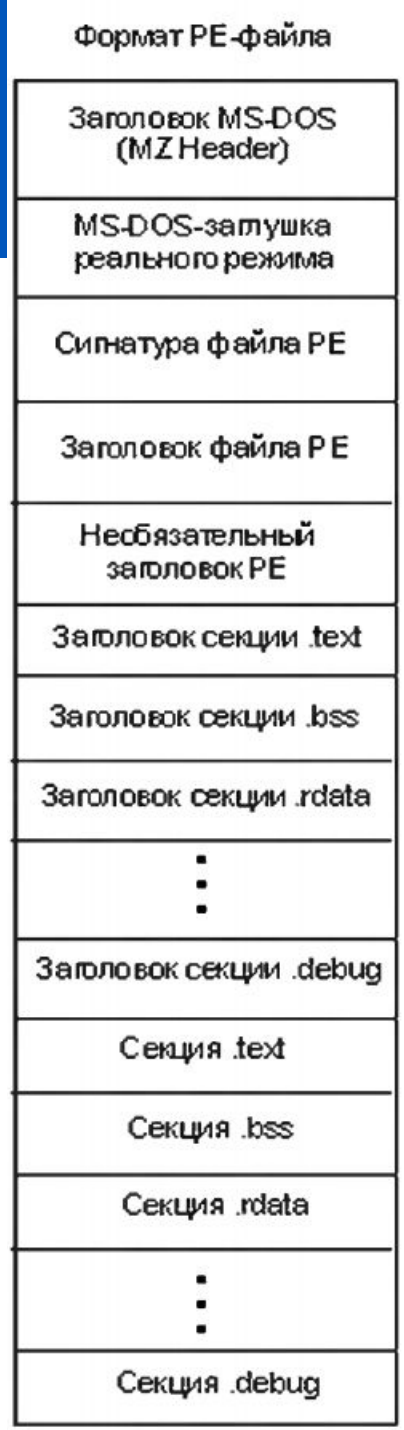
Основные секции:

1. **.text** - машинный код
2. **.rodata** - константы
3. **.data** - инициализированные глобальные переменные
4. **.bss** - неинициализированные глобальные переменные



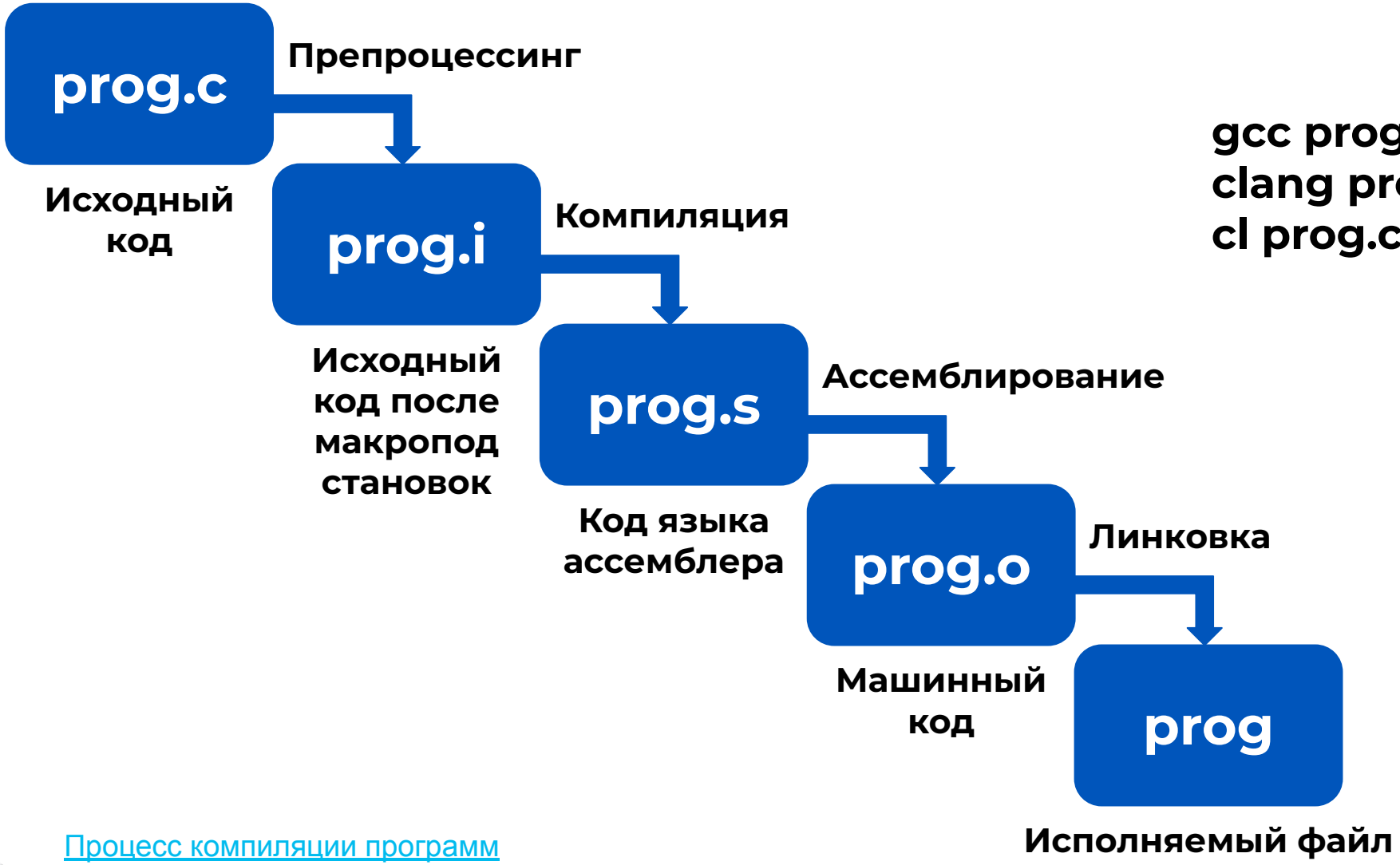
Введение в ELF-файлы в Linux: понимание и анализ / Хабр

Структура PE



[PE (Portable Executable): На странных берегах / Хабр]

Стадии компиляции




```
gcc prog.c -o prog
clang prog.c -o prog
cl prog.c (msvc)
```

Процесс компиляции программ на C++ / Хабр

1. Препроцессинг

gcc -E prog.c -o prog.i

1. Раскрываются *#include*
2. Раскрываются макросы *#define*
3. Удаляются комментарии
4. Условная компиляция (*#ifdef*, *#if*...)

2. Компиляция

gcc -S prog.i -o prog.s

1. Исходный код транслируется в ассемблерный
2. Компилятор может делать оптимизации (в зависимости от заданного уровня оптимизации)

3. Ассемблирование

gcc -c prog.s -o prog.o

1. Ассемблерный код транслируется в машинный
2. Создаётся объектный файл с машинным кодом, который ещё не готов к запуску

4. Линковка

gcc prog.o -o prog

1. Связываются объектные модули
2. Добавляются стандартные библиотеки
3. Создаётся полноценный исполняемый файл, который готов к запуску

Линковка:

- **Статическая** - всё в один бинарь. Больше размер, проще работать, можно нарушить лицензию
- **Динамическая** - бинарь берёт функции из библиотеки, которая лежит где-то в другом месте

При запуске: загрузчик читает таблицу и мапит сегменты в память

Отладочные символы

1. Компиляция с отладочными символами

gcc -g prog.c -o prog

2. Компиляция без отладочных символов

gcc prog.c -o prog

3. strip - удаляет символы, чтобы уменьшить размер

strip -s prog

Про отладочные символы

Декомпиляторы

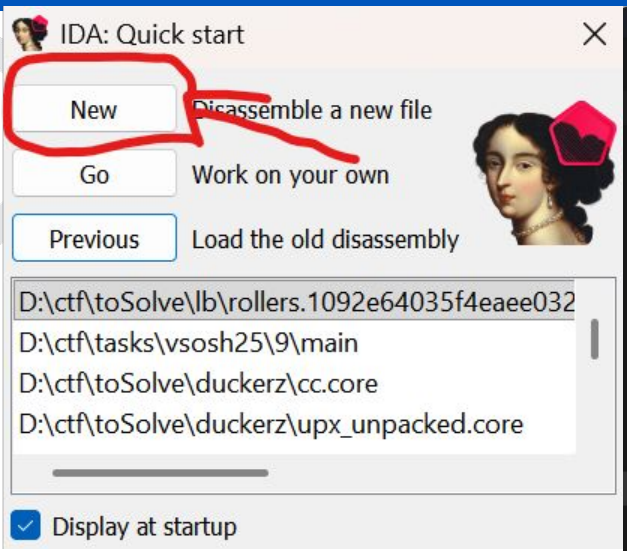
IDA

- Дорого
- Стабильно
- Удобно
- Поддерживает много что
- Много плагинов

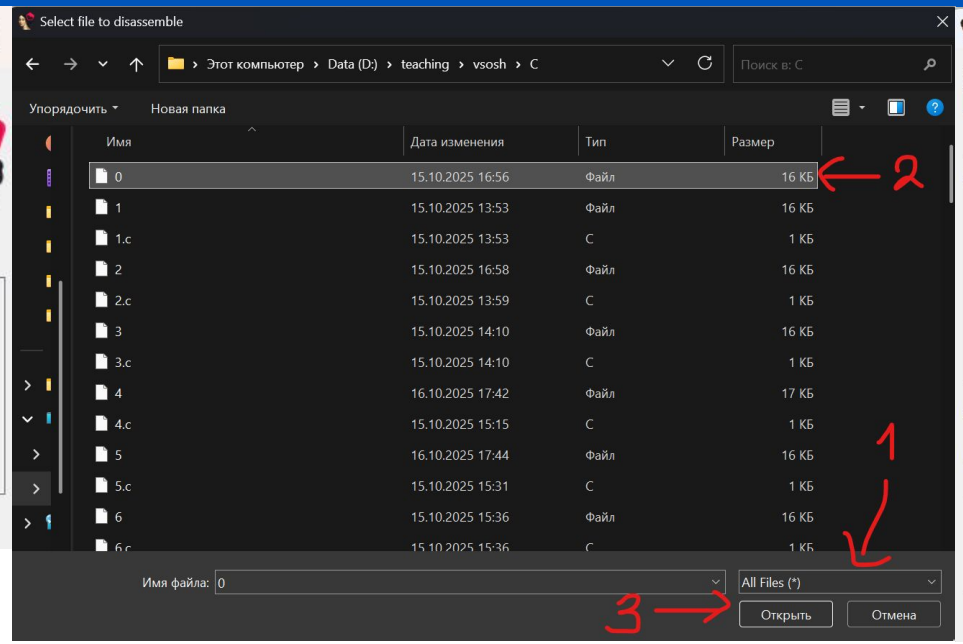
Ghidra

- Бесплатно, опенсорс
- Не так удобно
- Удобно делать свои плагины

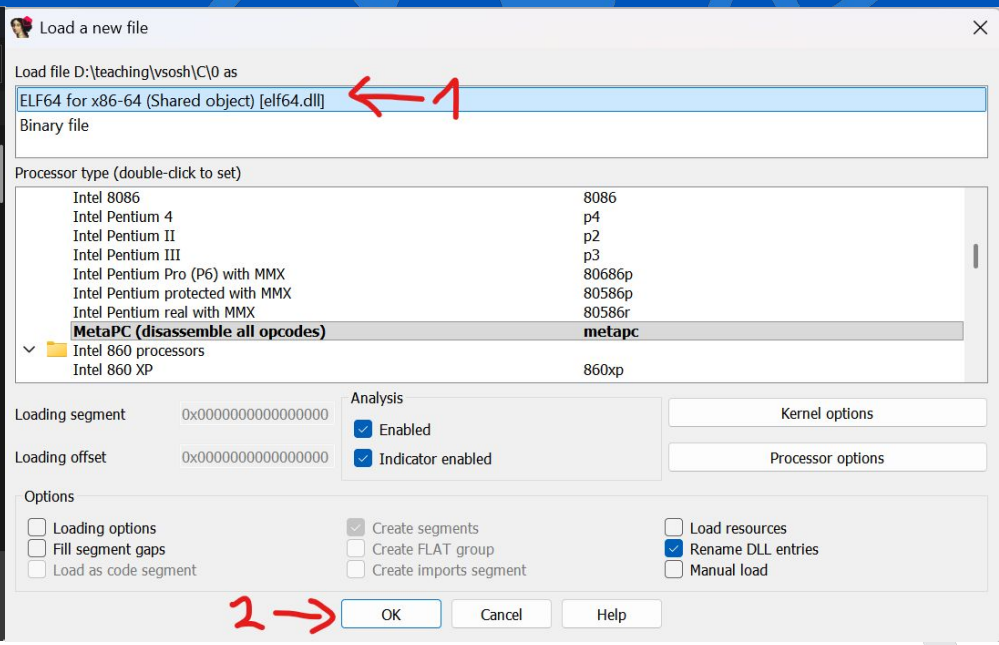
Загрузка бинаря



Открываем новый файл
(можно открыть существующий)



Выбираем отображение всех файлов, выбираем нужный и открываем его



Выбираем тип файла (обычно определяется автоматически)

Ассемблерный код в виде графа

Найденные функции

Рабочая зона с графом. Можно открыть другие окна: hex-редактор, таблица импортов, таблица экспортов

Консоль. Можно использовать IDAPython

Если нажать пробел, граф пропадёт

IDA - 0 D:\teaching\vsosh\C\0

File Edit Jump Search View Debugger Lumina Options Windows Help

No debugger

Library function | Regular function | Instruction | Data | Unexplored | External symbol | Lumina function

Functions

Function name	Segm
_init_proc	.init
sub_1020	.plt
sub_1030	.plt
__cxa_finalize	.plt.g
_printf	.plt.s
_start	.text
deregister_tm_clones	.text
register_tm_clones	.text
__do_global_dtors_aux	.text
frame_dummy	.text
square	.text
main	.text
_term_proc	.fini
__libc_start_main	exte
printf	exte
__imp___cxa_finalize	exte
__gmon_start__	exter

Line 16 of 17, /__imp___cxa_finalize

IDA View-A | Hex View-1 | Local Types | Imports | Exports

```
.text:000000000000115C
.text:000000000000115C ; =============== S U B R O U T I N E =======================================
.text:000000000000115C
.text:000000000000115C ; Attributes: bp-based frame
.text:000000000000115C
.text:000000000000115C ; int __fastcall main(int argc, const char **argv, const char **envp)
.text:000000000000115C                 public main
.text:000000000000115C main            proc near               ; DATA XREF: _start+18↑o
.text:000000000000115C
.text:000000000000115C var_8           = dword ptr -8
.text:000000000000115C var_4           = dword ptr -4
.text:000000000000115C
.text:000000000000115C ; __unwind {
.text:000000000000115C                 endbr64
.text:0000000000001160                 push    rbp
.text:0000000000001161                 mov     rbp, rsp
.text:0000000000001164                 sub     rsp, 10h
.text:0000000000001168                 mov     [rbp+var_8], 0
.text:000000000000116F                 mov     [rbp+var_4], 0
.text:0000000000001176                 jmp     short loc_1189
.text:0000000000001178 ; ---------------------------------------------------------------------------
.text:0000000000001178
.text:0000000000001178 loc_1178:                               ; CODE XREF: main+34↓j
.text:0000000000001178                 mov     eax, [rbp+var_4]
.text:000000000000117B                 mov     edi, eax
.text:000000000000117D                 call    square
.text:0000000000001182                 add     [rbp+var_8], eax
.text:0000000000001185                 add     [rbp+var_4], 1
.text:0000000000001189
.text:0000000000001189 loc_1189:                               ; CODE XREF: main+1A↑j
.text:0000000000001189                 cmp     [rbp+var_4], 3E7h
.text:0000000000001190                 jle     short loc_1178
```

0000115C 000000000000115C: main (Synchronized with Hex View-1)

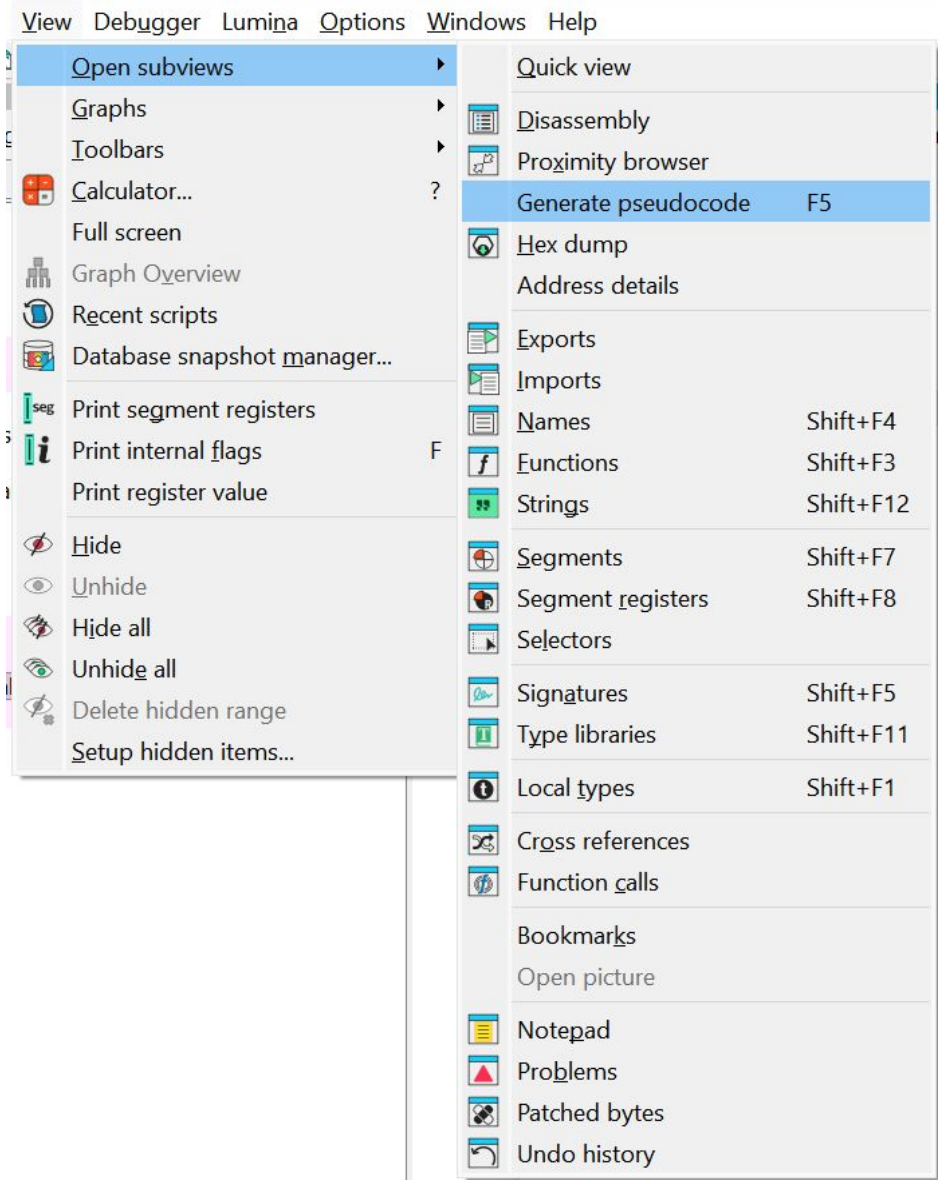
Output

```
Function argument information has been propagated
lumina: getaddrinfo: ???? ???? ??????????.
The initial autoanalysis has been finished.
-------------------------------------------------------------------------------
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun  6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)]
IDAPython 64-bit v9.0.0 final (serial 0) (c) The IDAPython Team <idapython@googlegroups.com>
-------------------------------------------------------------------------------
```

Python

AU: idle Up Disk: 44GB

Если нажать F5 или так, появится псевдокод



Псевдокод функции main

IDA - 0 D:\teaching\vsosh\C\0

File Edit Jump Search View Debugger Lumina Options Windows Help

Library function | Regular function | Instruction | Data | Unexplored | External symbol | Lumina function

Functions

Function name	Segment
_init_proc	.init
sub_1020	.plt
sub_1030	.plt
__cxa_finalize	.plt.g
_printf	.plt.s
_start	.text
deregister_tm_clones	.text
register_tm_clones	.text
__do_global_dtors_aux	.text
frame_dummy	.text
square	.text
main	.text
_term_proc	.fini
__libc_start_main	exte
printf	exte
__imp___cxa_finalize	exte
__gmon_start__	exter

Line 16 of 17, /__imp___cxa_finalize

IDA View-A | Pseudocode-A | Hex View-1 | Local Types | Imports | Exports

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  int v4; // [rsp+8h] [rbp-8h]
  int i; // [rsp+Ch] [rbp-4h]

  v4 = 0;
  for ( i = 0; i <= 999; ++i )
    v4 += square((unsigned int)i);
  printf("%d\n", v4);
  return 0;
}
```

0000115C main:1 (115C)

Output

```
lumina: getaddrinfo: ???? ???? ??????????.
The initial autoanalysis has been finished.
-------------------------------------------------------------------------------
Python 3.12.4 (tags/v3.12.4:8e8a4ba, Jun  6 2024, 19:30:16) [MSC v.1940 64 bit (AMD64)]
IDAPython 64-bit v9.0.0 final (serial 0) (c) The IDAPython Team <idapython@googlegroups.com>
-------------------------------------------------------------------------------
1149: using guessed type __int64 __fastcall square(_QWORD);
```

Python

AU: idle Up Disk: 44GB

Основные возможности

1. **N** - переименовать переменную или функцию
2. **Ctrl + F** - Различный поиск
3. **Y** - изменить тип функции или переменной
4. **Кликнуть 2 раза** - перейти к коду функции или расположению переменной
5. **** - скрыть приведения типов
6. **/** - оставить комментарий
7. **X** - посмотреть, где используется
8. **View -> Subviews -> Strings** - показать найденные строки

IDA View-A | Pseudocode-A | Strings

Address	Length	Type	String
LOAD:0000...	0000001C	C	/lib64/ld-linux-x86-64.so.2
LOAD:0000...	0000000F	C	__cxa_finalize
LOAD:0000...	00000012	C	__libc_start_main
LOAD:0000...	00000007	C	printf
LOAD:0000...	0000000A	C	libc.so.6
LOAD:0000...	0000000C	C	GLIBC_2.2.5
LOAD:0000...	0000000B	C	GLIBC_2.34
LOAD:0000...	0000001C	C	_ITM_deregisterTMCloneTable
LOAD:0000...	0000000F	C	__gmon_start__
LOAD:0000...	0000001A	C	_ITM_registerTMCloneTable
.eh_frame:0...	00000006	C	:*3$\"

Основы IDAPython

1. **here() или get_screen_ea()** - адрес, на котором курсор

```
Python>get_screen_ea()
0x116f
Python>here()
0x116f
```

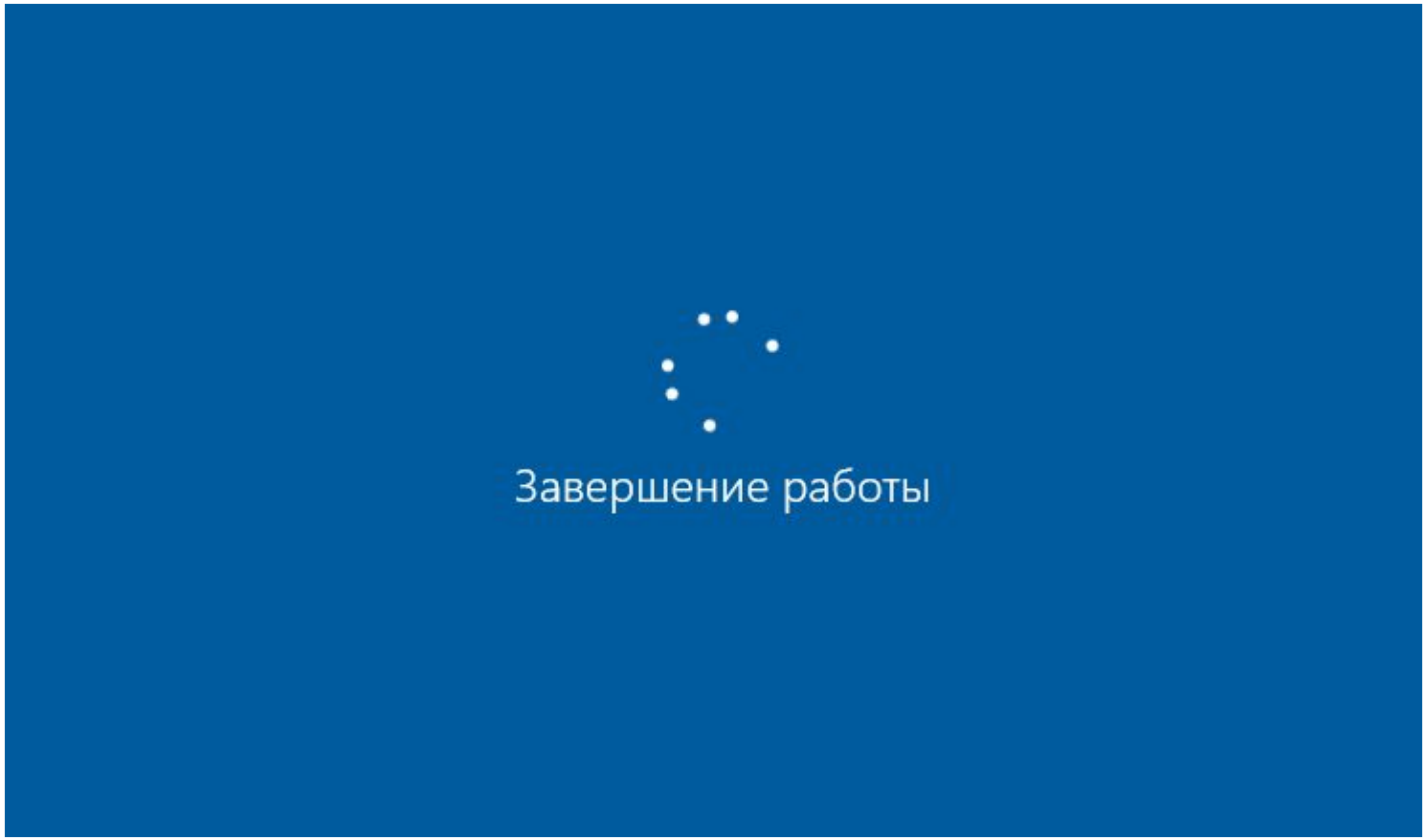
Python

2. **get_bytes(addr, cnt)** - получить cnt байтов, начиная с адреса addr (Аналогично для других размеров)
3. **get_name(addr)** - имя по переданному адресу
4. **print_insn_mnem(addr)** - мнемоника инструкции по переданному адресу

И так далее...

Завершение работы

При завершении работы можно сохранить текущие результаты в базу данных Ida и при необходимости отправить на другой компьютер



Итоги

НИЯУ МИФИ

1. Рассмотрены основные подходы к реверс-инжинирингу
2. Рассмотрен статический анализ исходного кода
3. Рассмотрен статический анализ Python, C#, Java
4. Рассмотрены типы исполняемых файлов, стадии компиляции
5. Рассмотрен статический анализ исполняемых файлов в Ida Pro

Ответы на вопросы

Полезные материалы

1. Tasks :: forkbomb.ru - видео и задачи для тренировки
2. Flare-On - шикарный CTF для ревёрсеров. Сложность постепенно растёт от первой до последней задачи
3. play.picoctf.org - хороший сайт с задачами
4. Введение в реверсинг с нуля, используя IDA PRO - курс по реверсу
5. К. Касперски “Искусство дизассемблирования” - старая, но хорошая книга.
6. Сикорски М., Хониг Э. “Вскрытие покажет! Практический анализ вредоносного ПО” - хорошая книга про анализ ВПО
7. Chris Eagle “The IDA Pro Book: The Unofficial Guide to the World’s Most Popular Disassembler”

Спасибо
за внимание!